ORM y pickle juntos

Ejercicio práctico paso a paso: la base de datos de una tienda y su copia de seguridad

En este ejercicio vas a guardar los clientes de una tienda en una base de datos usando un ORM. Después harás una copia de seguridad con pickle, borrarás todos los clientes como si hubiera sido un accidente y los recuperarás desde la copia.

Sigue los pasos en orden. Cada paso tiene el código que tienes que escribir y un pantallazo de Visual Studio Code con lo que debes ver en tu pantalla.

1. Lo que ya sabes

- **pickle** guarda objetos de Python en un **archivo**. Con `pickle.dump` se guardan y con `pickle.load` se recuperan.
- Un **ORM** (SQLAlchemy) guarda objetos de Python en una **base de datos**, convirtiendo cada clase en una tabla. Tú trabajas con objetos y el ORM escribe el SQL.

2. Cómo se unen en este ejercicio

- La **base de datos** (con el ORM) es el sitio principal donde viven los clientes de la tienda.
- El archivo de **pickle** es una **copia de seguridad**: una foto de los clientes en un momento concreto, por si algo sale mal.

Momento	Qué pasa
Copia de seguridad	Base de datos → ORM → lista → pickle → copia.bin
Restaurar	copia.bin → pickle → lista → ORM → base de datos

En medio usaremos una **lista** normal de Python. El ORM saca los clientes de la base de datos y los pone en la lista; pickle coge esa lista y la guarda en el archivo. Para restaurar se hace el camino al revés.

3. Los archivos del ejercicio

Archivo	Qué hace	Usa
`modelos.py`	Define la clase Cliente y crea la tabla	ORM
`crear.py`	Guarda tres clientes	ORM
`leer.py`	Muestra los clientes de la base de datos	ORM
`copia.py`	Hace la copia de seguridad en copia.bin	ORM y pickle
`borrar_todo.py`	Borra todos los clientes (el accidente)	ORM
`restaurar.py`	Recupera los clientes desde la copia	ORM y pickle

Ejecuta cada archivo **una sola vez** y en el orden del ejercicio. Si te equivocas y quieres empezar de cero, borra los archivos `tienda.db` y `copia.bin` y vuelve al paso 4.

4. Antes de empezar: sobre los pantallazos

- Los pantallazos están hechos con Visual Studio Code **en inglés**. Si el tuyo está en español, los menús se llaman igual pero traducidos: **File** es **Archivo**, **Open Folder** es **Abrir carpeta**, **New Terminal** es **Nuevo terminal**.
- En la terminal de los pantallazos, cada orden empieza por `alumno@clase:~/tienda_copia$`. En Windows verás algo parecido a `PS C:\Users\tu_nombre\tienda_copia>`. No importa: lo que escribes después es lo mismo.
- En Mac, escribe `python3` en lugar de `python` en todas las órdenes.

Paso 1. Instalar Python y Visual Studio Code

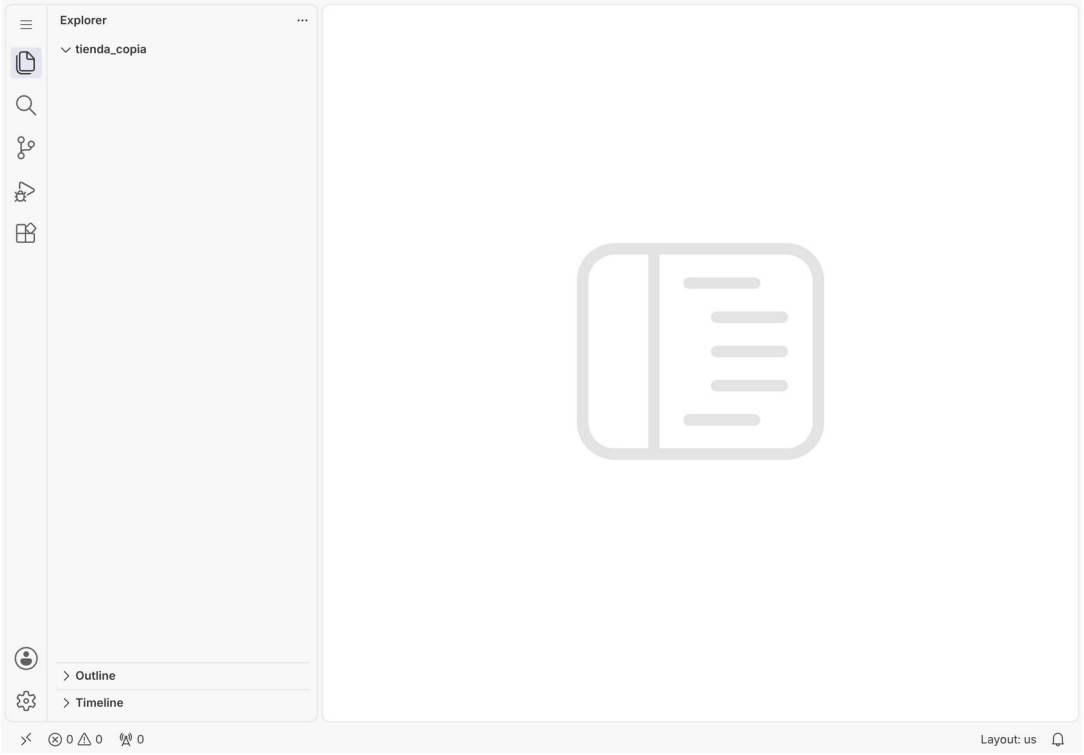
1. Descarga Python desde **python.org** (apartado Downloads) y ejecuta el instalador. En Windows, marca la casilla **Add python.exe to PATH** antes de pulsar Install Now. Sin esa casilla, la terminal no encontrará Python.
2. Descarga e instala **Visual Studio Code** desde **code.visualstudio.com**. Es gratuito.

pickle no hay que instalarlo: viene incluido con Python. SQLAlchemy lo instalaremos en el paso 3.

Paso 2. Abrir la carpeta y la terminal

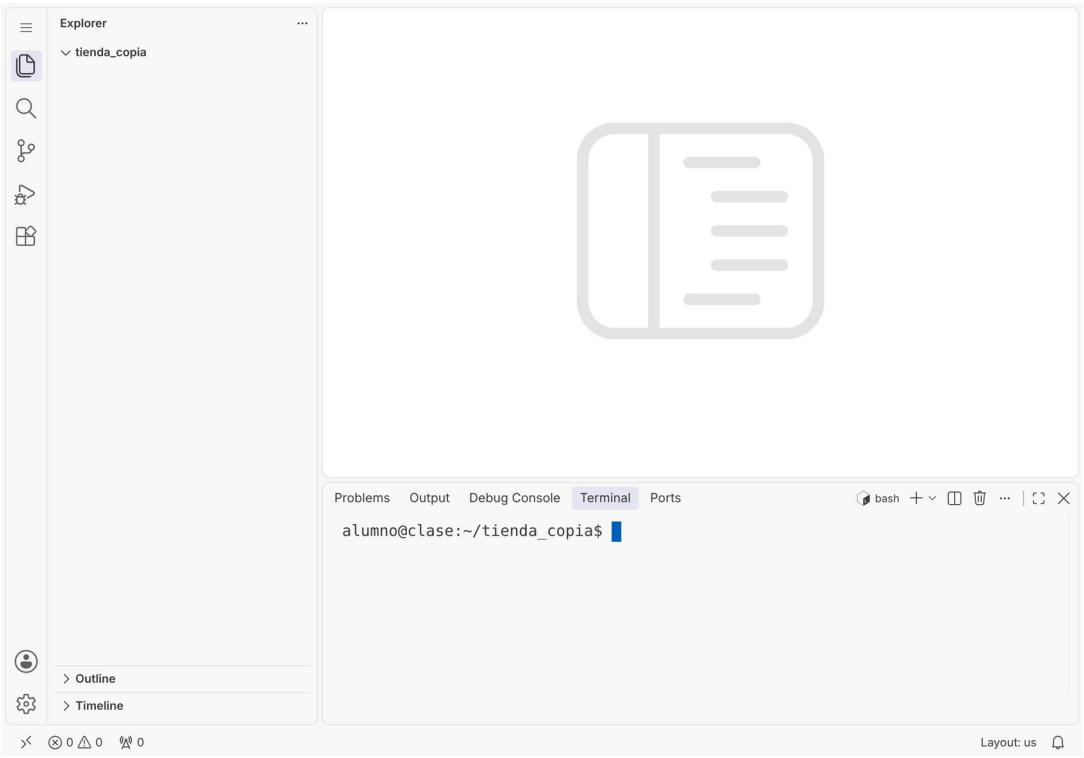
1. Crea en tu ordenador una carpeta nueva llamada **tienda_copia**, por ejemplo en Documentos.
2. Abre Visual Studio Code. Entra en el menú **File** y elige **Open Folder**. Selecciona la carpeta tienda_copia.

A la izquierda, en el panel **Explorer**, aparece el nombre de tu carpeta. Todavía está vacía:



Captura 1. La carpeta tienda_copia abierta en Visual Studio Code

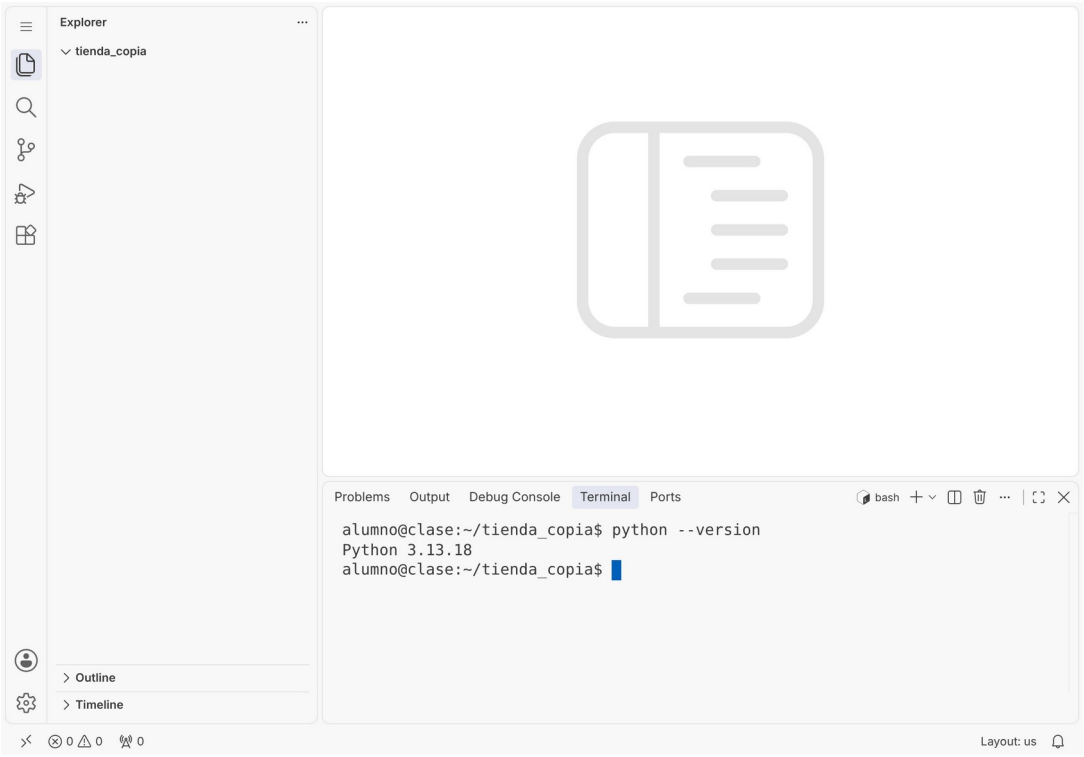
3. Abre la terminal desde el menú **Terminal** y elige **New Terminal**. Aparece en la parte de abajo, ya situada dentro de tu carpeta:



Captura 2. La terminal abierta en la parte de abajo

4. Comprueba que Python está bien instalado. Escribe en la terminal esta orden y pulsa Intro:

```
python --version
```



Captura 3. Python responde con su versión

Tu número puede ser distinto; lo importante es que empiece por **Python 3**.

Paso 3. Instalar SQLAlchemy

En la terminal escribe esta orden y pulsa Intro:

```
python -m pip install sqlalchemy
```

`pip` es el instalador de Python: descarga SQLAlchemy de internet. Espera hasta que aparezca un mensaje que empieza por **Successfully installed**. Los números de versión pueden ser distintos en tu ordenador.



Captura 4. SQLAlchemy instalado correctamente

Paso 4. La base de datos: modelos.py

Para crear un archivo, entra en **File**, elige **New File**, escribe el nombre `modelos.py` y pulsa Intro. Copia dentro este código, respetando los espacios del principio de las líneas:

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import declarative_base

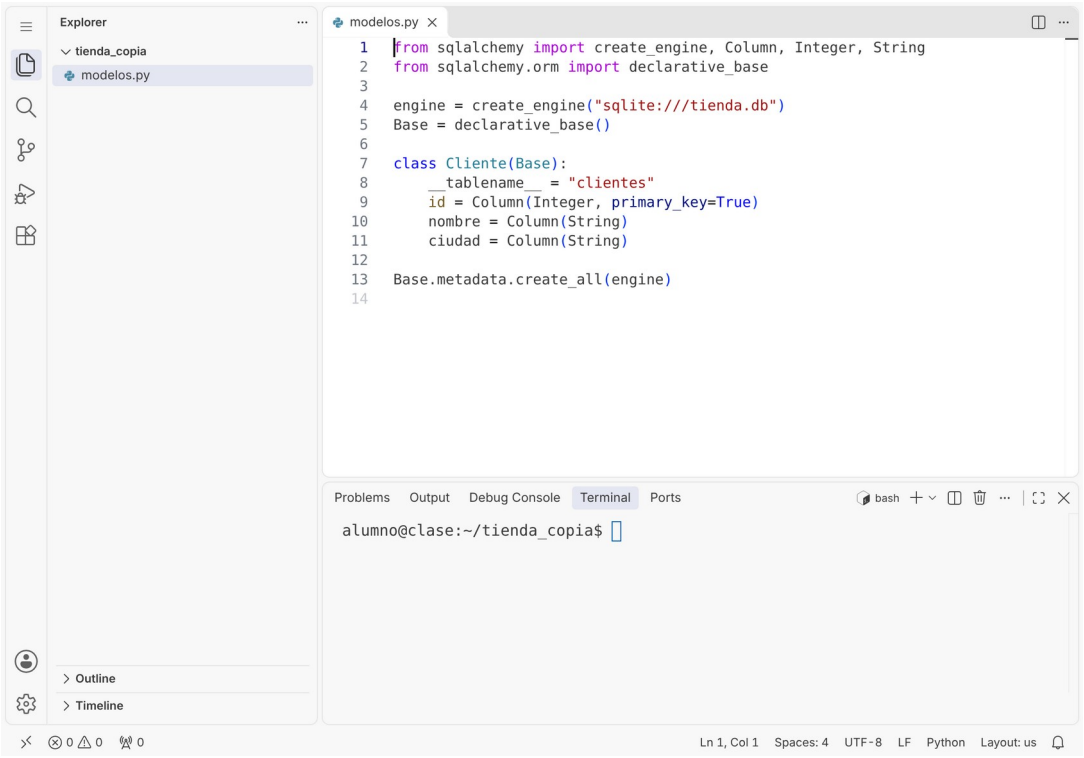
engine = create_engine("sqlite:///tienda.db")
Base = declarative_base()

class Cliente(Base):
    __tablename__ = "clientes"
    id = Column(Integer, primary_key=True)
```

```
    nombre = Column(String)
    ciudad = Column(String)

Base.metadata.create_all(engine)
```

Guarda el archivo con **Ctrl + S** (en Mac, Cmd + S). Debe quedarte así:



Captura 5. El archivo modelos.py en el editor

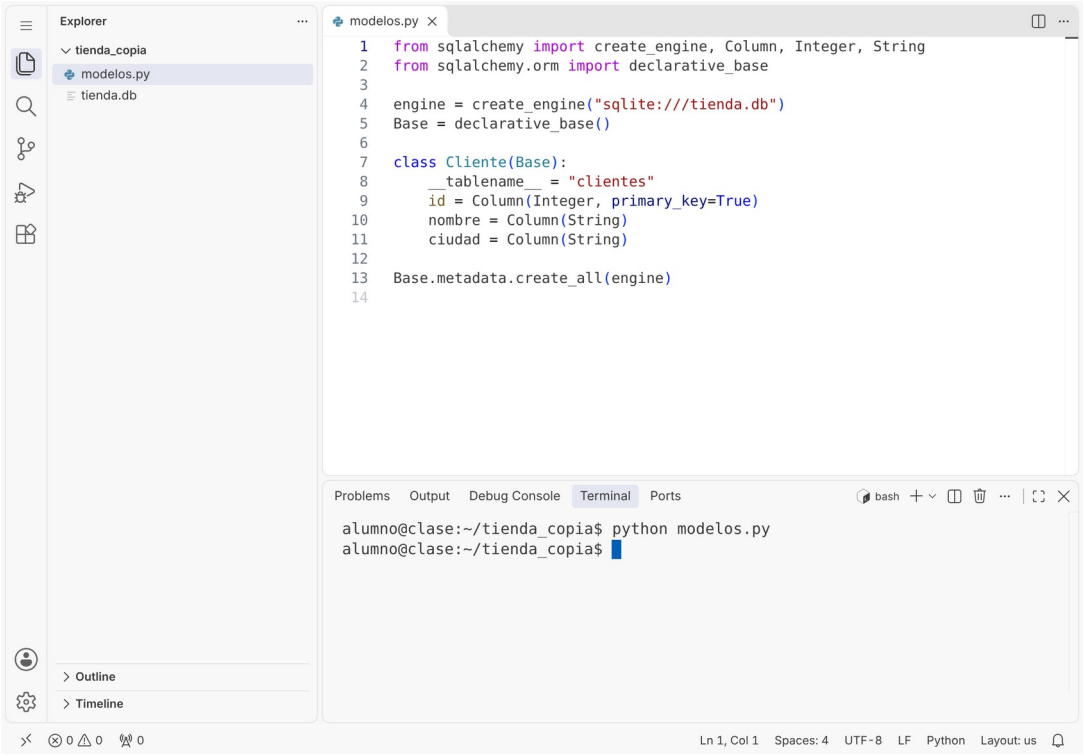
Qué hace

- `create_engine` prepara la conexión con la base de datos, que se guardará en el archivo **tienda.db**.
- La clase **Cliente** se convierte en la tabla **clientes**. Cada `Column` es una columna: el `id` (un número que pone el ORM solo), el nombre y la ciudad.
- `create_all` crea la tabla en la base de datos.

Ejecutarlo

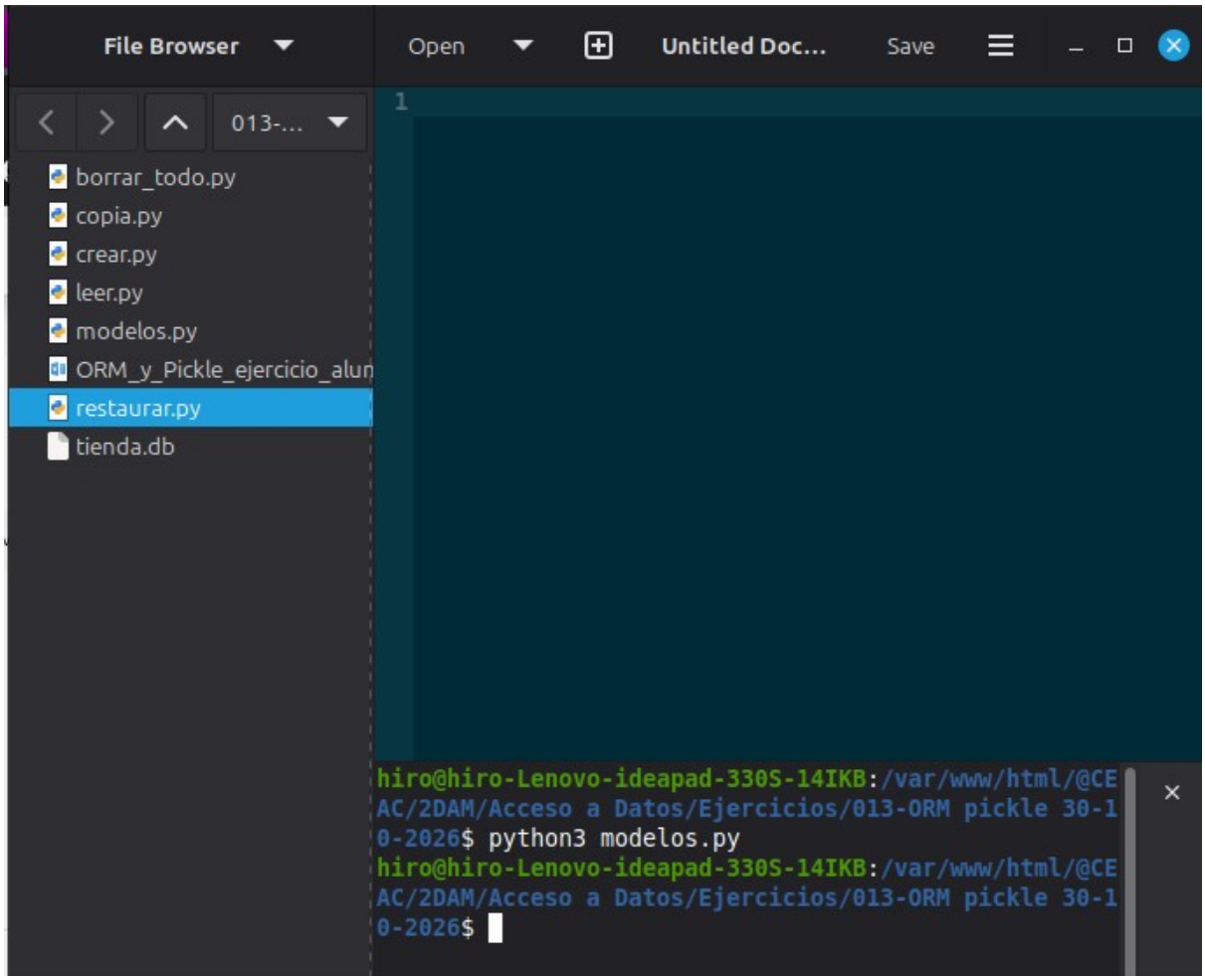
Escribe en la terminal:

```
python modelos.py
```



Captura 6. Ha aparecido el archivo tienda.db

La terminal no muestra ningún mensaje, y es normal: este archivo no tiene ningún `print`. Pero fíjate en el panel **Explorer**: ha aparecido **tienda.db**. Esa es tu base de datos, con la tabla clientes dentro.



Paso 5. Guardar clientes: crear.py

Crea el archivo `crear.py` con este código y guárdalo:

```
from sqlalchemy.orm import Session
from modelos import Cliente, engine

with Session(engine) as session:
    ana = Cliente(nombre="Ana", ciudad="Valencia")
    luis = Cliente(nombre="Luis", ciudad="Madrid")
    marta = Cliente(nombre="Marta", ciudad="Valencia")

    session.add_all([ana, luis, marta])
    session.commit()

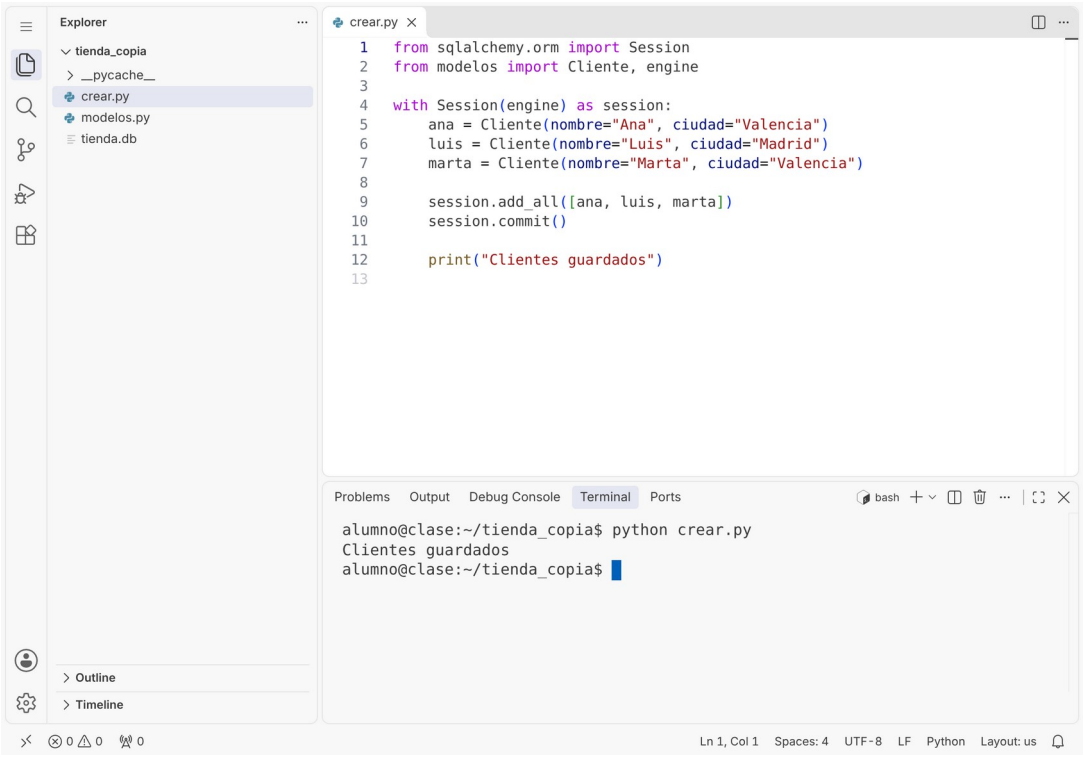
    print("Clientes guardados")
```

Qué hace

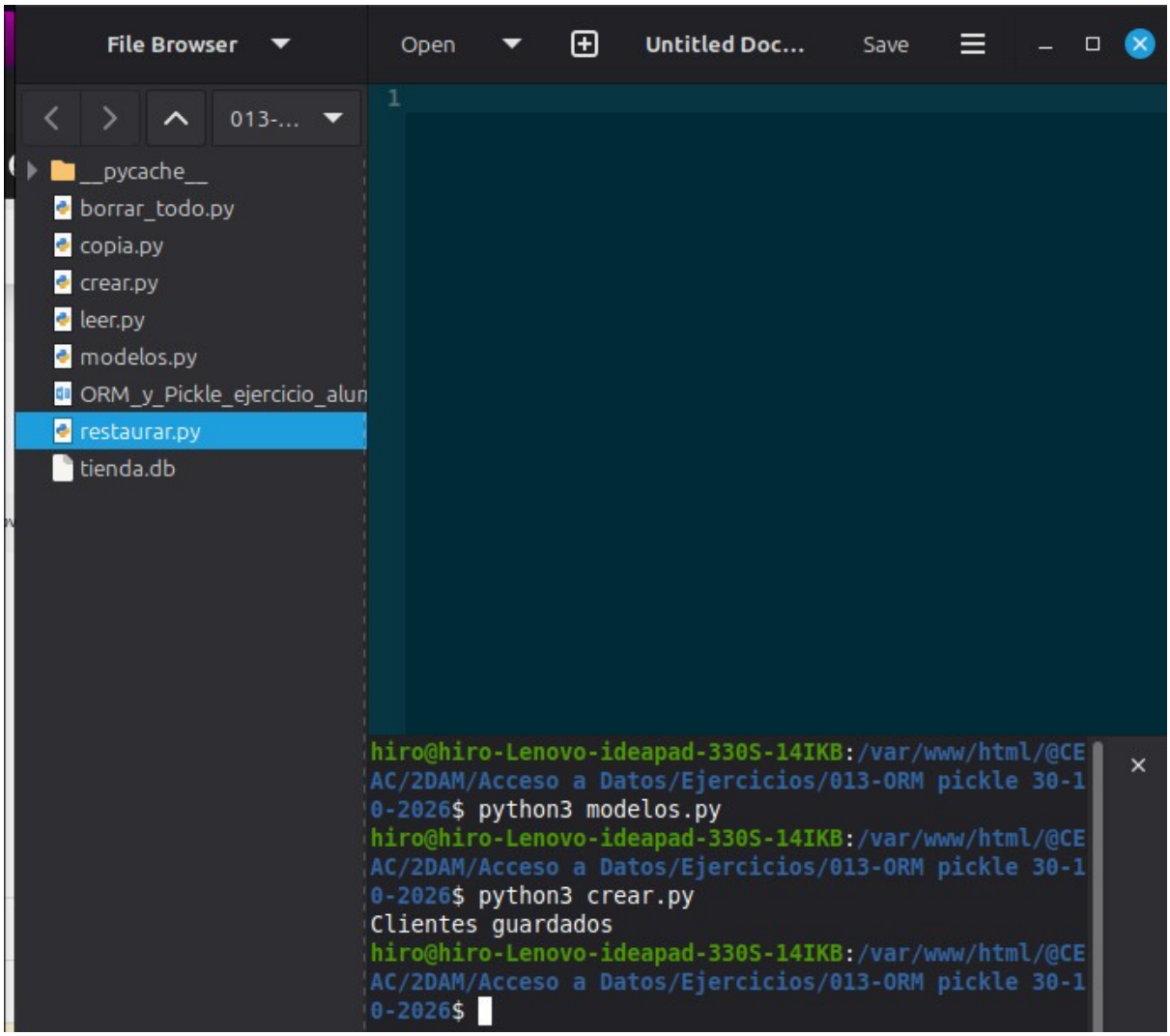
- `with Session(engine) as session:` abre una **sesión**, que es la conversación con la base de datos.
- Las tres líneas siguientes crean tres objetos Cliente: Ana, Luis y Marta. No les damos `id`: lo pondrá el ORM.
- `add_all` prepara los tres clientes y `commit` los guarda de verdad en la base de datos.

Ejecútalo:

```
python crear.py
```



Captura 7. Los clientes se han guardado



Paso 6. Ver los clientes: leer.py

Vamos a necesitar ver varias veces qué hay en la base de datos, así que hacemos un programa solo para eso. Crea `leer.py`:

```
from sqlalchemy.orm import Session
from modelos import Cliente, engine

with Session(engine) as session:
    clientes = session.query(Cliente).all()
    print("Clientes en la base de datos:", len(clientes))
    for cliente in clientes:
        print(cliente.id, cliente.nombre, cliente.ciudad)
```

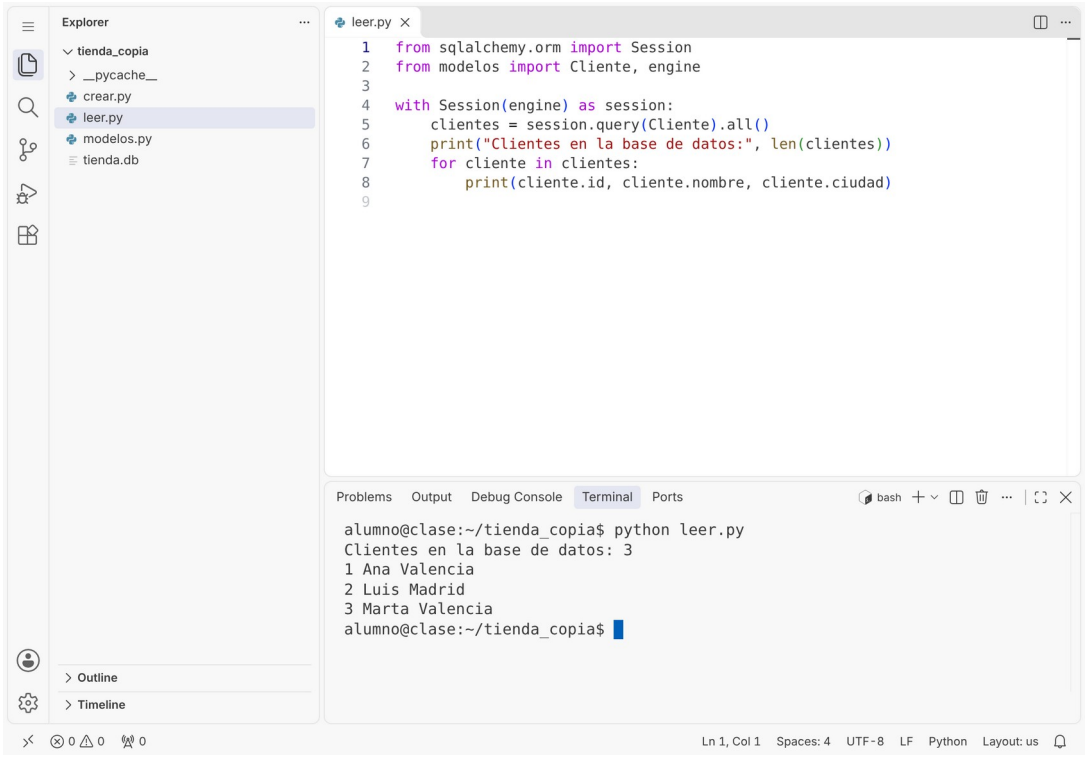
Qué hace

- `session.query(Cliente).all()` pide al ORM **todos** los clientes y los guarda en la lista clientes.

- `len(clientes)` cuenta cuántos hay. Así veremos enseguida si la base de datos está llena o vacía.
- El `for` recorre la lista y muestra el número, el nombre y la ciudad de cada cliente.

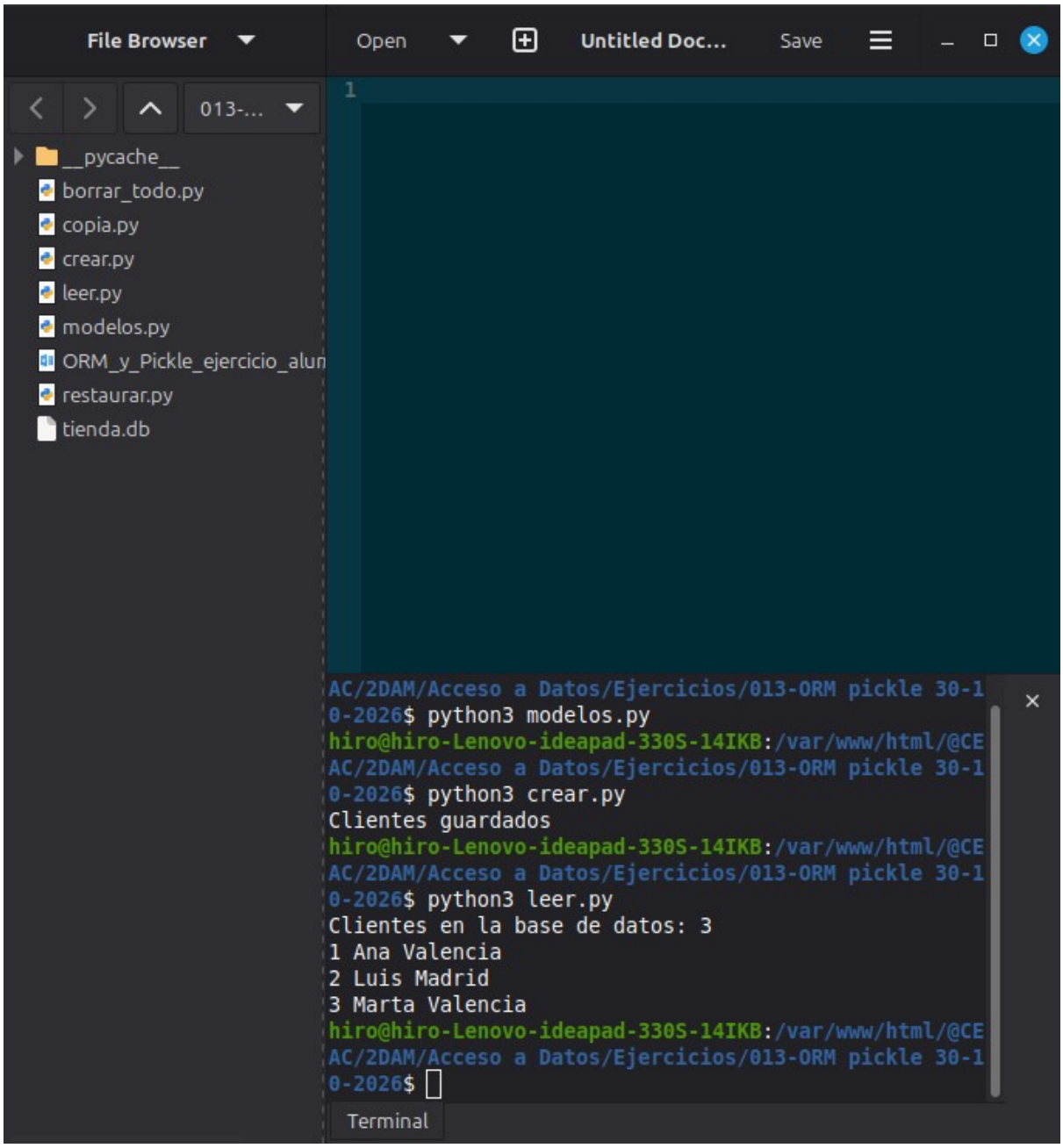
Ejecútalo:

```
python leer.py
```



Captura 8. Hay tres clientes en la base de datos

Cada cliente tiene ya su número: 1, 2 y 3. Lo ha puesto el ORM.



Paso 7. Hacer la copia de seguridad: copia.py

Este es el primer programa que **une el ORM y pickle**: el ORM lee los clientes de la base de datos y pickle los guarda en un archivo. Crea `copia.py`:

```
import pickle
from sqlalchemy.orm import Session
from modelos import Cliente, engine

with Session(engine) as session:
    clientes = session.query(Cliente).all()
```

```
    datos = []
    for cliente in clientes:
        datos.append({"nombre": cliente.nombre, "ciudad":
cliente.ciudad})

archivo = open("copia.bin", "wb")
pickle.dump(datos, archivo)
archivo.close()

print("Copia guardada con", len(datos), "clientes")
```

Qué hace cada parte

```
clientes = session.query(Cliente).all()
```

Parte del ORM. Lee todos los clientes de la base de datos, igual que en leer.py.

```
datos = []
```

Crea una lista vacía donde vamos a meter los datos de cada cliente.

```
datos.append({"nombre": cliente.nombre, "ciudad": cliente.ciudad})
```

Para cada cliente, mete en la lista una pequeña **ficha** con su nombre y su ciudad. Lo que va entre llaves `{ }` se llama **diccionario**: una ficha con etiquetas ("nombre", "ciudad") y un valor para cada una.

No guardamos directamente los objetos del ORM porque están conectados a la base de datos. Es más sencillo y seguro copiar solo los datos.

```
archivo = open("copia.bin", "wb")
```

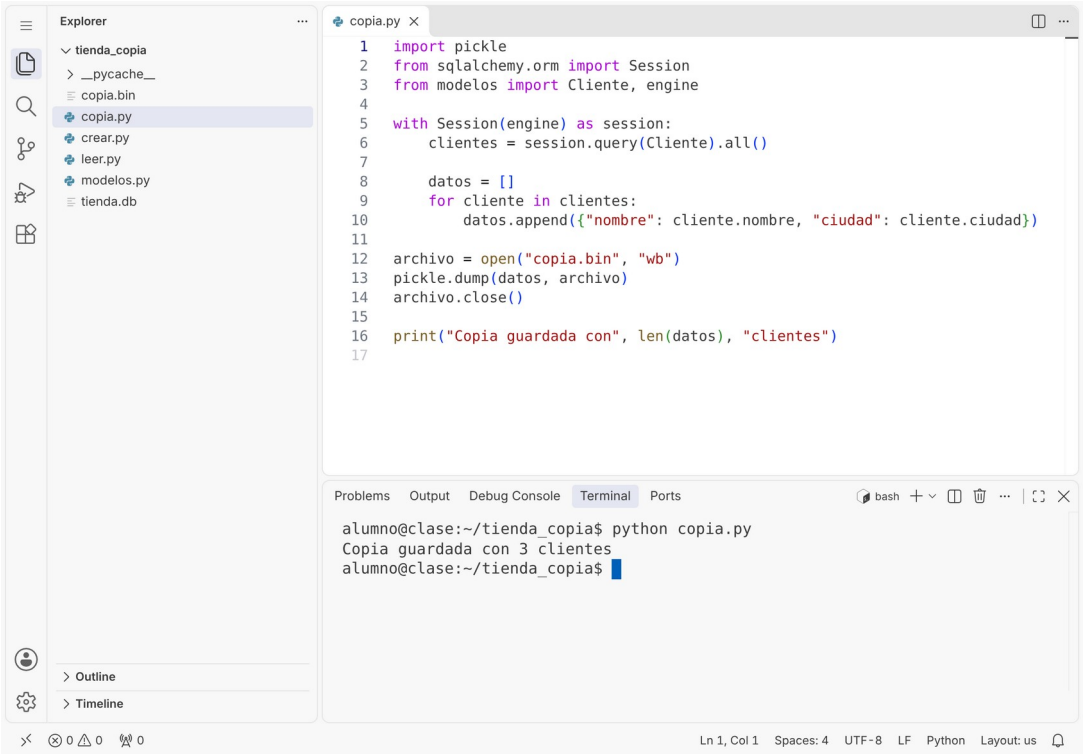
Parte de pickle. Abre el archivo copia.bin para escribir (**w**) en modo binario (**b**).

```
pickle.dump(datos, archivo)
```

Guarda la lista con las tres fichas en el archivo. Después, `archivo.close()` lo cierra.

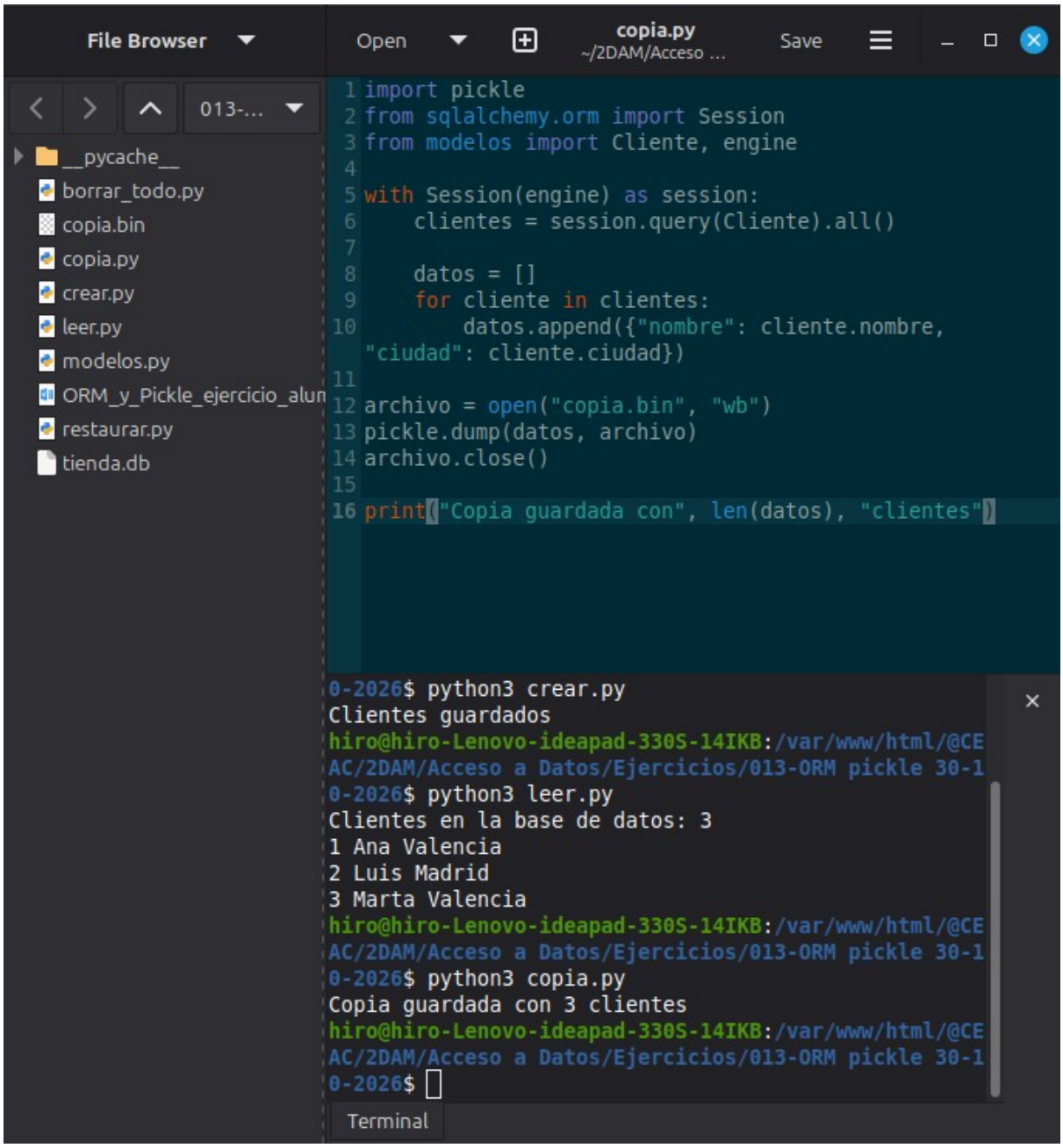
Ejecútalo:

```
python copia.py
```



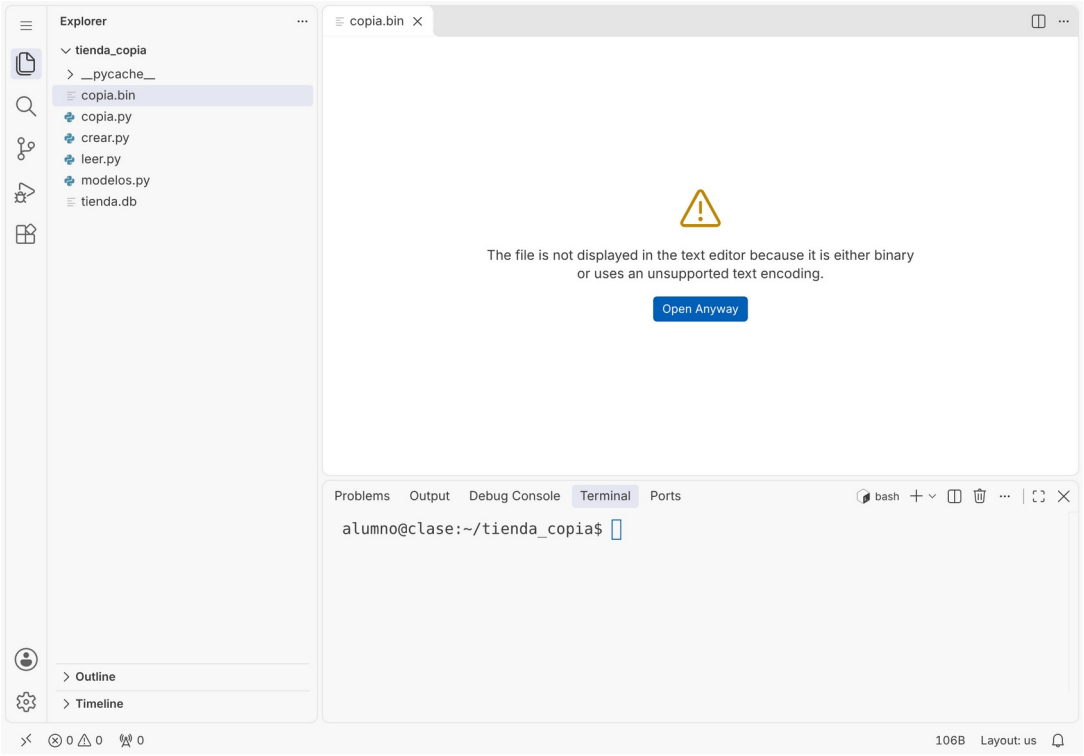
Captura 9. La copia se ha hecho y ha aparecido copia.bin

En el panel **Explorer** ahora hay **dos** archivos de datos: **tienda.db**, la base de datos que maneja el ORM, y **copia.bin**, la copia de seguridad que maneja pickle.



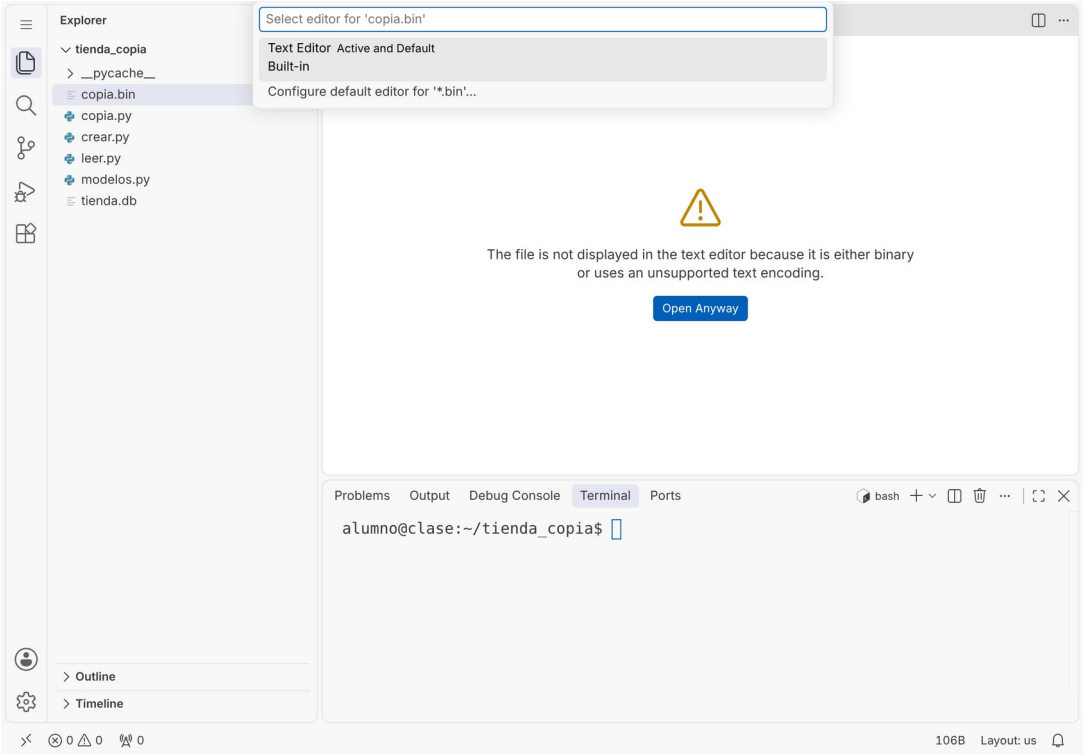
Paso 8. Mirar qué hay dentro de copia.bin

Haz clic en **copia.bin** en el panel Explorer. Visual Studio Code avisa de que es un archivo binario y no lo muestra:



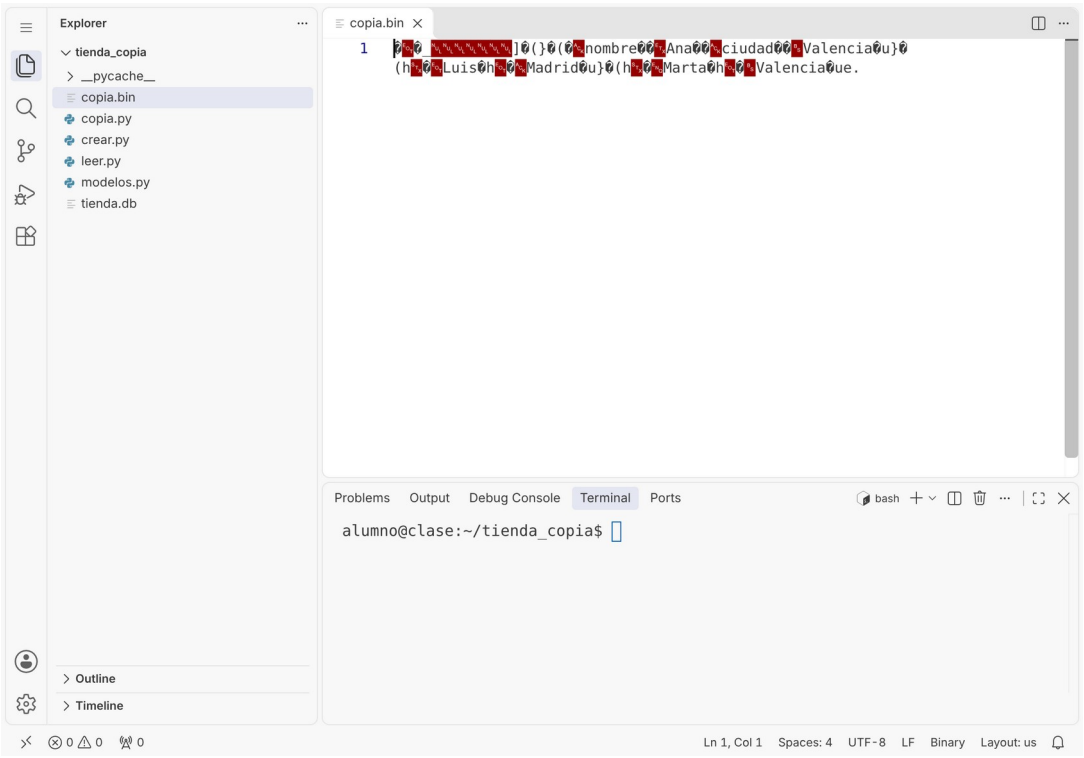
Captura 10. Aviso de archivo binario

Pulsa el botón **Open Anyway** (abrir de todos modos). Arriba aparece una lista para elegir cómo abrirlo. Elige **Text Editor** y pulsa Intro:



Captura 11. Elegir Text Editor

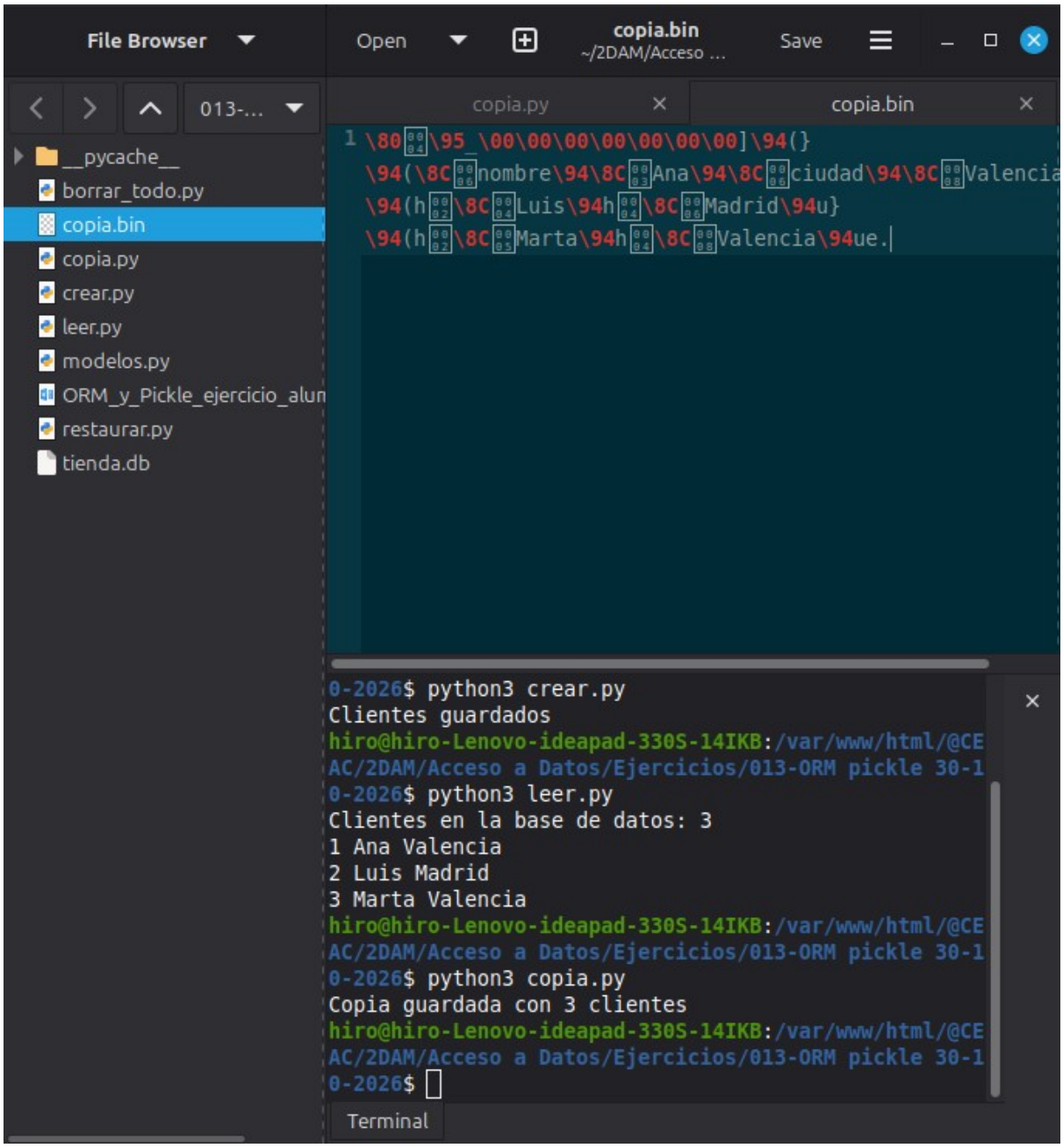
Ahora sí se ve el contenido:



Captura 12. El contenido de copia.bin

Aparecen símbolos raros y recuadros rojos: son instrucciones internas de pickle que no son letras. Entre ellos se leen las etiquetas **nombre** y **ciudad** y los datos de los tres clientes: **Ana**, **Valencia**, **Luis**, **Madrid**, **Marta**...

Cierra este archivo sin modificarlo ni guardarlo. Si cambias algo, pickle ya no podrá leerlo.



Paso 9. El accidente: borrar_todo.py

Vamos a imaginar que alguien se equivoca y borra todos los clientes. Crea `borrar_todo.py`:

```
from sqlalchemy.orm import Session
from modelos import Cliente, engine

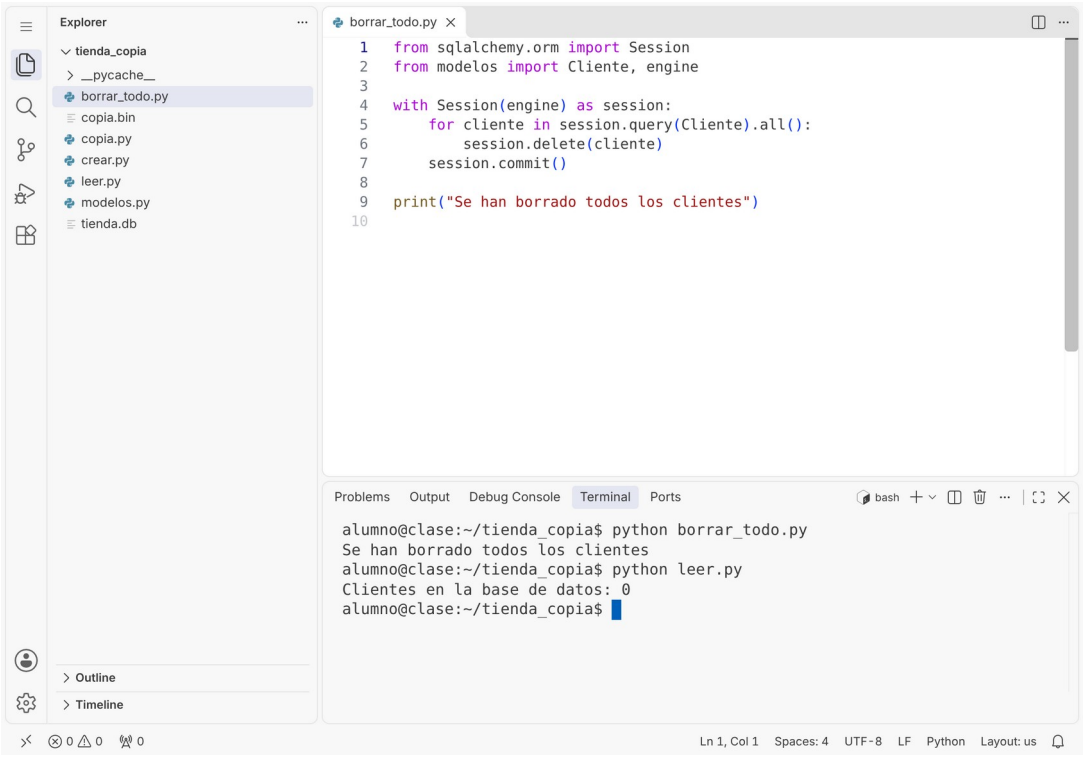
with Session(engine) as session:
    for cliente in session.query(Cliente).all():
        session.delete(cliente)
    session.commit()
```

```
print("Se han borrado todos los clientes")
```

El `for` recorre todos los clientes, `session.delete` prepara el borrado de cada uno y `session.commit()` lo confirma.

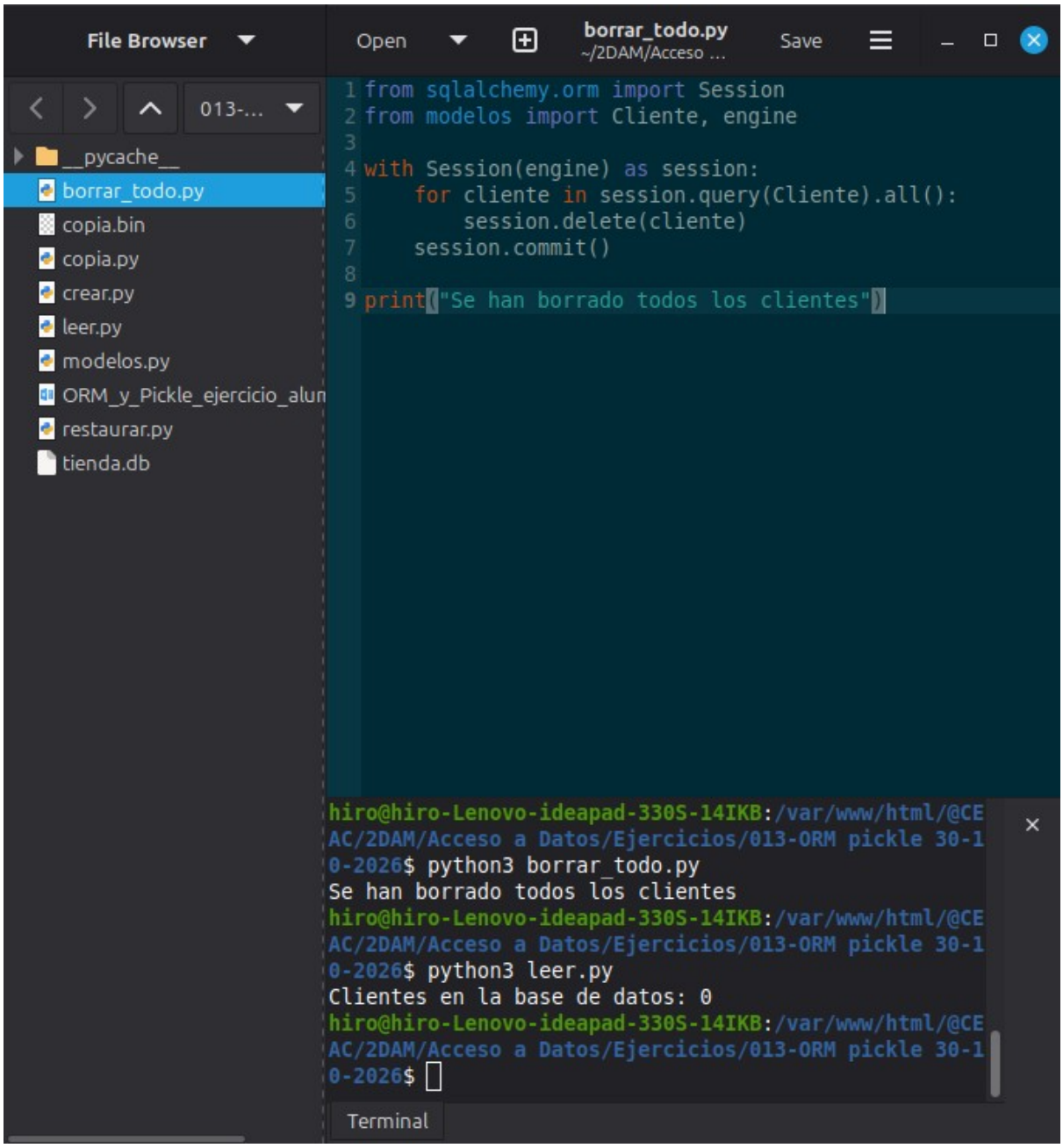
Ejecútalo y comprueba después con leer.py cómo ha quedado la base de datos:

```
python borrar_todo.py
python leer.py
```



Captura 13. La base de datos se ha quedado vacía

La base de datos tiene **0 clientes**. Sin copia, los datos se habrían perdido. Pero copia.bin sigue en la carpeta con los tres clientes, porque el ORM solo ha tocado tienda.db.



Paso 10. Recuperar los datos: restaurar.py

Este es el segundo programa que **une pickle y el ORM**, ahora en sentido contrario: pickle lee la copia y el ORM vuelve a meter los clientes en la base de datos. Crea `restaurar.py`:

```
import pickle
from sqlalchemy.orm import Session
from modelos import Cliente, engine


archivo = open("copia.bin", "rb")
datos = pickle.load(archivo)
```

```
archivo.close()

with Session(engine) as session:
    for d in datos:
        session.add(Cliente(nombre=d["nombre"], ciudad=d["ciudad"]))
    session.commit()

print("Restaurados", len(datos), "clientes")
```

Qué hace cada parte

```
archivo = open("copia.bin", "rb")
```

Parte de pickle. Abre la copia para leer (**r**) en modo binario (**b**).

```
datos = pickle.load(archivo)
```

Lee el archivo y recupera la lista con las tres fichas. Después, `archivo.close()` lo cierra.

```
for d in datos:
```

Parte del ORM. Recorre las fichas una a una. En cada vuelta, `d` es la ficha de un cliente.

```
session.add(Cliente(nombre=d["nombre"], ciudad=d["ciudad"]))
```

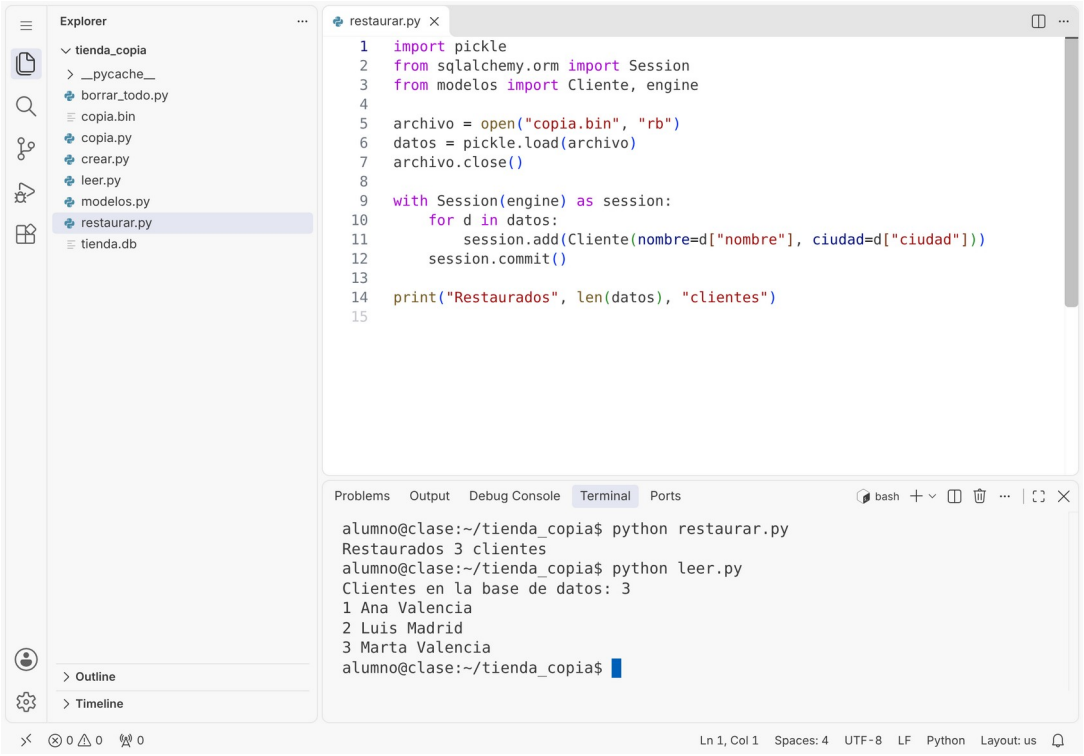
Crea un objeto Cliente nuevo con los datos de la ficha y lo prepara para guardarlo. `d["nombre"]` significa «el valor de la etiqueta nombre de esta ficha».

```
session.commit()
```

Guarda los clientes en la base de datos.

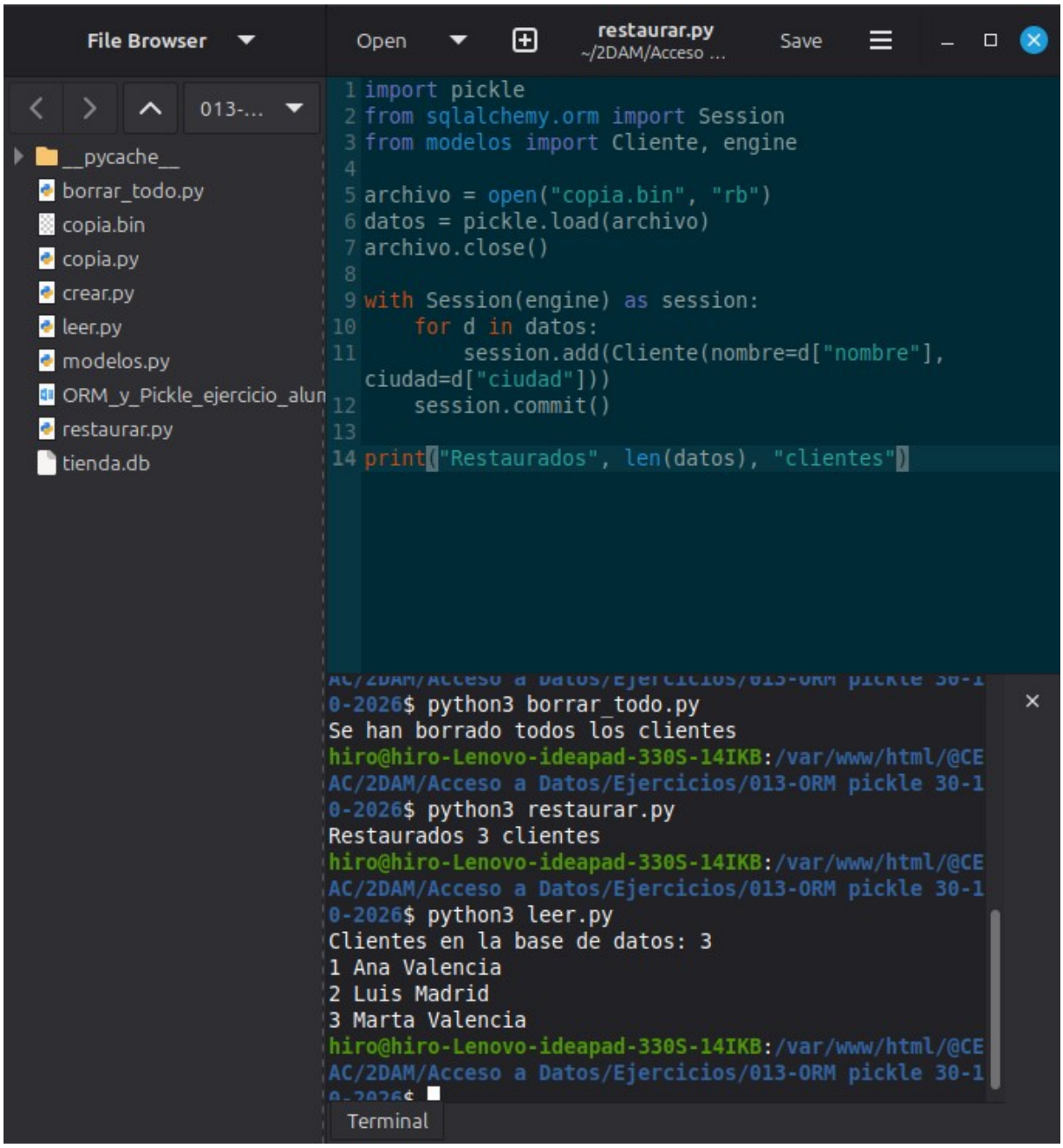
Ejecútalo y comprueba el resultado con leer.py:

```
python restaurar.py
python leer.py
```



Captura 14. Los clientes han vuelto a la base de datos

Los tres clientes vuelven a estar en la base de datos, con sus nombres y sus ciudades. La copia de seguridad ha funcionado.


File Browser
Open
restaurar.py
~/2DAM/Acceso ...
Save
013-...
__pycache__
borrar_todo.py
copia.bin
copia.py
crear.py
leer.py
modelos.py
ORM_y_Pickle_ejercicio_alun
restaurar.py
tienda.db
1 import pickle
2 from sqlalchemy.orm import Session
3 from modelos import Cliente, engine
4
5 archivo = open("copia.bin", "rb")
6 datos = pickle.load(archivo)
7 archivo.close()
8
9 with Session(engine) as session:
10 for d in datos:
11 session.add(Cliente(nombre=d["nombre"],
ciudad=d["ciudad"]))
12 session.commit()
13
14 print("Restaurados", len(datos), "clientes")
0-2026$ python3 borrar_todo.py
Se han borrado todos los clientes
hiro@hiro-Lenovo-ideapad-330S-14IKB:/var/www/html/@CE
AC/2DAM/Acceso a Datos/Ejercicios/013-ORM pickle 30-1
0-2026$ python3 restaurar.py
Restaurados 3 clientes
hiro@hiro-Lenovo-ideapad-330S-14IKB:/var/www/html/@CE
AC/2DAM/Acceso a Datos/Ejercicios/013-ORM pickle 30-1
0-2026$ python3 leer.py
Clientes en la base de datos: 3
1 Ana Valencia
2 Luis Madrid
3 Marta Valencia
hiro@hiro-Lenovo-ideapad-330S-14IKB:/var/www/html/@CE
AC/2DAM/Acceso a Datos/Ejercicios/013-ORM pickle 30-1
Terminal

Resumen

Estas son las piezas que has usado y de qué herramienta viene cada una:

Pieza	Para qué sirve	Viene de
`class Cliente(Base)`	La clase que se convierte en tabla	ORM
`session.query(...).all()`	Leer todos los clientes	ORM
`session.add(...)`	Preparar un cliente para guardarlo	ORM
`session.delete(...)`	Preparar el borrado de un cliente	ORM
`session.commit()`	Confirmar los cambios en la base de datos	ORM
`{"nombre": ...}`	Una ficha con los datos de un cliente	Python
`open(..., "wb")`	Abrir un archivo para escribir	Python
`open(..., "rb")`	Abrir un archivo para leer	Python
`pickle.dump(...)`	Guardar la lista en el archivo	pickle
`pickle.load(...)`	Recuperar la lista del archivo	pickle

La idea que debes quedarte es esta: **el ORM y pickle no compiten, se complementan**. El ORM se encarga de la base de datos, donde se trabaja cada día. pickle guarda una copia de esos datos en un archivo aparte. Para unirlos solo hace falta una lista de Python en medio: el ORM la llena y pickle la guarda, o pickle la recupera y el ORM la vuelve a meter en la base de datos.